1. System Overview

You are building a **Distributed Cluster Scheduler** with:

- **Central Control Plane (Scheduler)**
- **Distributed Data Plane (Docker Nodes)**
- **State Store (Postgres)**
- **Observability Layer (Dashboard + Metrics)**

Supports:

- Normal nodes (stable servers)
- Mobile-like nodes (unreliable, low compute, high churn)

---

2. Core Components

🔹 1. API Gateway / Interface Layer

Handles:

- CLI requests
- REST API calls
- Dashboard backend

Responsibilities:

- Accept job submissions
- Query cluster state
- Trigger actions (scale, reschedule)

🔹 2. Scheduler (Control Plane Brain)

Responsibilities:

- Watch pending tasks
- Select nodes using pluggable strategies
- Assign tasks
- Handle rescheduling on failures

Subcomponents:

- Task Queue
- Scheduling Engine (pluggable)
- Node Filter
- Node Scorer
- Binding Engine

🔹 3. Node Manager (Control Plane)

Responsibilities:

- Track node health (heartbeat)
- Maintain node registry
- Detect failures

🔹 4. Worker Node Agent (Data Plane)

Runs on each node (Docker container or process)

Responsibilities:

- Register with scheduler
- Send heartbeats
- Execute tasks (Docker containers)
- Report status

🔹 5. Task Execution Engine (Inside Node)

Responsibilities:

- Run Docker containers
- Monitor resource usage
- Report completion/failure

🔹 6. State Store (Postgres)

Stores:

- Nodes
- Tasks
- Allocations
- Metrics

🔹 7. Metrics & Analytics Engine (unique feature)

Responsibilities:

- Resource utilization tracking
- Scheduling efficiency
- Fragmentation analysis
- Node reliability scoring (important for mobile nodes)

🔹 8. Adaptive Node Profiler (UNIQUE FEATURE)

Handles:

- Unpredictable nodes
- Low compute nodes
- Dynamic availability

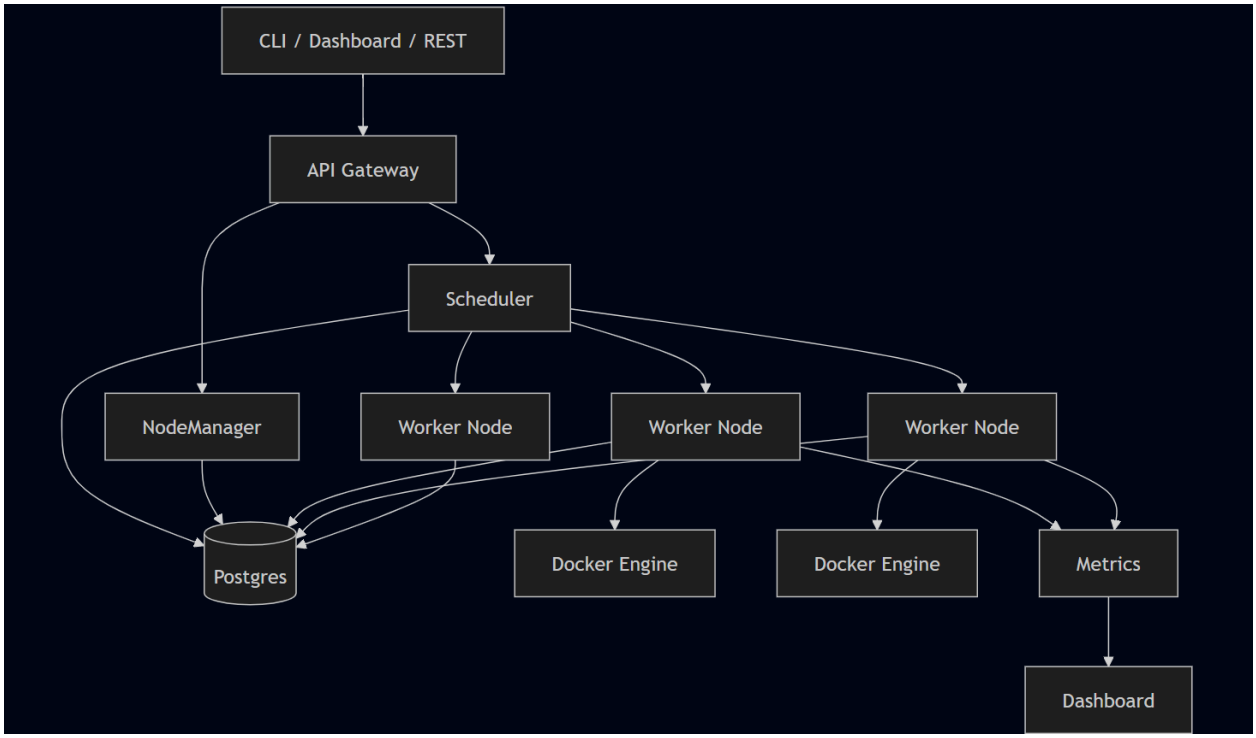
Tracks:

- uptime %
- failure rate
- average execution success

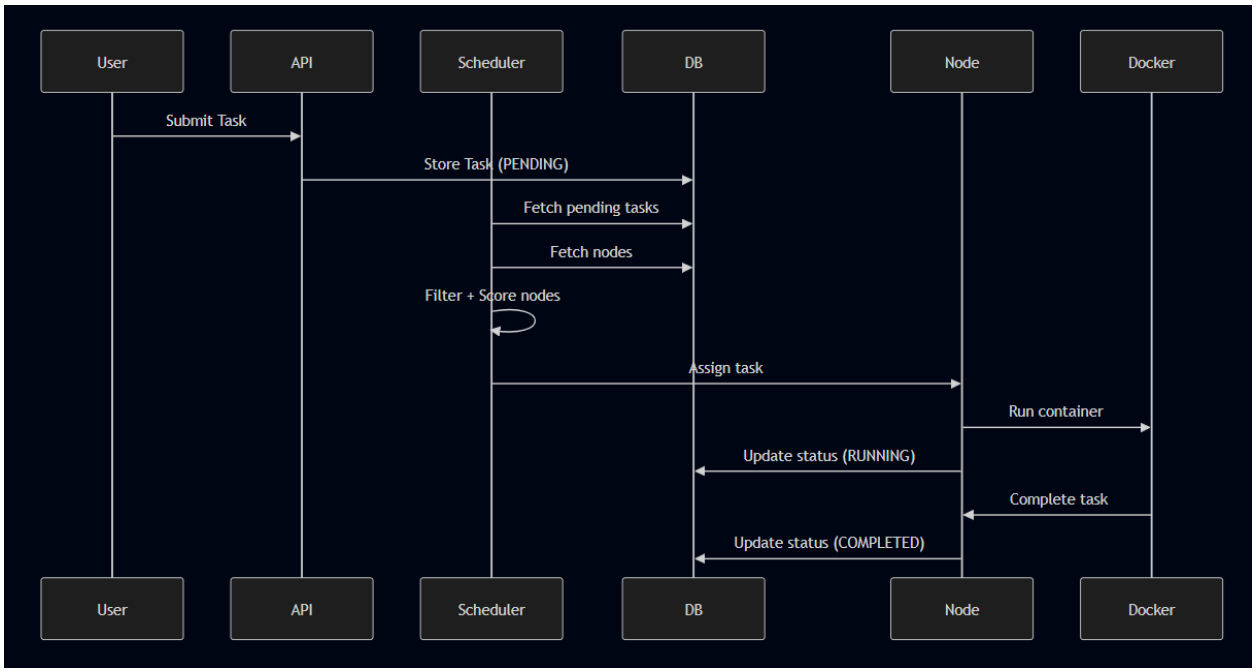
Used in scheduling decisions.

---

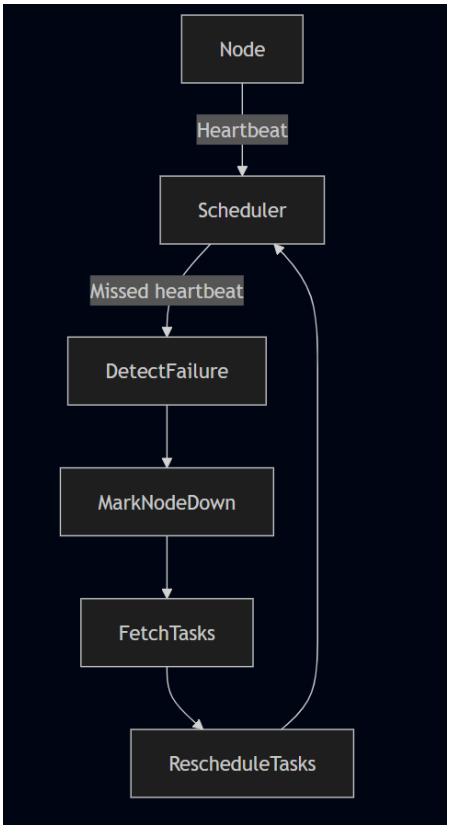
3. Architecture Diagram



4. Task Scheduling Flow



5. Failure Handling Flow



6. Scheduling Pipeline (K8s-style)

1. Filter nodes (resource + availability)
2. Score nodes:
   - resource fit
   - reliability score
   - fragmentation
3. Select best node
4. Bind task

---

7. Special Feature: Unpredictable Numerous Nodes

Handled via:

- Reliability score
- Dynamic weighting in scheduler
- Lower priority for critical tasks
- Suitable for:
    - batch jobs
    - low priority workloads